

ISTVT: Interpretable Spatial-Temporal Video Transformer for Deepfake Detection

Rahul Rajesh Hegde
240831

Sanidhya Singh
240932

Shourya Mathur
240991

Tanishq Kundara
241088

EE656 COURSE PROJECT

Abstract—Deepfake detection has become an important research problem due to the rapid advancement of AI-generated media. In this project, we implement the Interpretable Spatial-Temporal Video Transformer (ISTVT), a transformer-based framework that leverages both spatial artifacts and temporal inconsistencies for robust deepfake detection. We selected this paper because it combines an effective video transformer architecture with an interpretable attention mechanism, making it both technically challenging and practically relevant. Our implementation focuses on reproducing the proposed methodology using a subset of the FaceForensics++ (C23) dataset, with the objective of understanding its architecture, evaluating its performance, and analyzing the interpretability of its predictions.

I. INTRODUCTION

The rapid advancement of deep learning has enabled the creation of highly realistic synthetic media, commonly referred to as deepfakes. While these techniques have found applications in entertainment and digital content creation, they also pose significant challenges in areas such as misinformation, identity fraud, and digital forensics. As deepfake generation methods continue to improve, developing reliable and robust detection techniques has become an increasingly important area of research.

Early deepfake detection methods primarily relied on analyzing individual image frames for visual artifacts. However, modern manipulation techniques often eliminate many of these artifacts, making frame-based approaches less effective. Since videos inherently contain temporal information, analyzing the consistency of facial movements and expressions across consecutive frames can provide additional cues for detecting manipulated content. Transformer-based architectures have recently demonstrated strong capabilities in modeling long-range spatial and temporal dependencies, making them well suited for video-based deepfake detection.

In this project, we implement and study the Interpretable Spatial-Temporal Video Transformer (ISTVT) proposed for deepfake detection. The model combines an Xception-based feature extractor with a spatial-temporal transformer and a self-subtract mechanism to capture both spatial artifacts and temporal inconsistencies. Our objective is to reproduce the proposed framework using a subset of the FaceForensics++ (C23) dataset, evaluate its performance, and gain a deeper understanding of its architecture and interpretability through implementation and experimentation.

II. DATA/DATASET

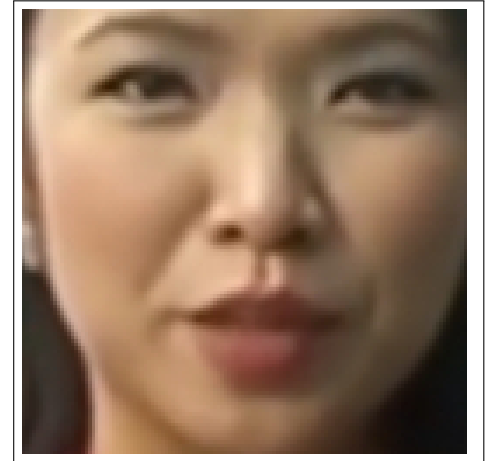
The experiments were conducted using the FaceForensics++ (FF++) C23 dataset obtained from Kaggle. The dataset consists of pristine videos together with manipulated videos generated using several state-of-the-art facial manipulation techniques. For this project, a representative subset of the dataset was selected, consisting of four deepfake generation techniques, namely *DeepFakeDetection*, *Face2Face*, *FaceSwap*, and *NeuralTextures*. Along with these manipulated videos, the corresponding pristine (original) videos were also included to form the real class. The initial composition of the selected dataset is summarized in Table I.

TABLE I: Initial Composition of the Dataset

Sl.No	Manipulation Technique	No. of Videos
1	DeepFakeDetection	60
2	Face2Face	60
3	NeuralTextures	60
4	FaceSwap	60
5	None (Real/Original)	100



(a) Original Size



(b) Final Cropped (128 x 128)

Fig. 1: Before vs After

The preprocessing pipeline consisted of three stages. First, `download_ff_subset.py` was used to obtain the required subset of the FaceForensics++ C23 dataset. Next, `preprocess.py` performed face detection and extraction using the **MTCNN** framework, resized the detected facial regions to 128×128 pixels, uniformly sampled six representative frames from each video, and stored them as PyTorch (.pt) tensors together with their corresponding labels and metadata. Finally, `filter_dataset.py` balanced the dataset by reducing the number of manipulated clips while retaining all original clips. The resulting dataset contained **6000** manipulated clips and **4500** original clips, providing a balanced and computationally efficient dataset for transformer training. The final dataset composition is presented in Table II.

TABLE II: Final Composition After Preprocessing

Sl.No	Description	No. of PT Files
1	Real (Original)	4500
2	Fake (All Manipulation Techniques)	6000



Fig. 2: (a) Original Video



Fig. 3: (b) Used Manipulation Technique: FaceSwap

Fig. 4: Real vs Fake Images

III. FEATURE EXTRACTION

A. Xception Backbone

The feature extraction stage was implemented using the `architecture/backbone.py` module. Following the methodology proposed in the ISTVT framework, only the Entry Flow of the Xception network was employed as the backbone feature extractor. Each of the six facial frames was processed independently to obtain high-level spatial representations while maintaining computational efficiency. The backbone converts an input clip of dimensions $6 \times 3 \times 128 \times 128$ into a feature tensor of dimensions $6 \times 728 \times 8 \times 8$, which is subsequently forwarded to the tokenizer module. All Xception Entry Flow parameters were frozen during training via `requires_grad=False`, ensuring only the tokenizer, transformer blocks, and classification head were updated during backpropagation.

B. Feature Tokenization

Feature tokenization was implemented in `architecture/tokenizer.py`. Each 8×8 feature map produced by the Xception backbone was interpreted as 64 spatial patches, which were flattened and linearly projected into a transformer embedding space. A learnable CLS token was appended to every frame sequence, increasing the number of tokens per frame from 64 to 65. To preserve structural and temporal information, learnable spatial positional embeddings and temporal positional embeddings were incorporated before the token sequence was passed to the temporal-spatial transformer encoder. The tokenizer therefore transforms the backbone output from $6 \times 728 \times 8 \times 8$ into a sequence of $6 \times 65 \times 512$ transformer tokens.

C. Implementation Modifications

Although the proposed implementation closely follows the original ISTVT architecture, several practical modifications were introduced to accommodate the computational limitations of the available hardware. The original implementation processes 300×300 facial images with a 768-dimensional transformer embedding. In contrast, the present implementation utilizes 128×128 facial inputs and a reduced 512-dimensional embedding space, significantly lowering GPU memory consumption while preserving the overall architecture and data flow. Furthermore, the paper does not explicitly describe the final aggregation strategy for frame-level CLS tokens. Therefore, the CLS tokens corresponding to all six frames were concatenated and passed through a multilayer perceptron for binary classification, thereby retaining temporal information learned across the complete clip.

IV. TRANSFORMER ARCHITECTURE

The transformer architecture was implemented in `architecture/transformer.py`, comprising 2 components: a position-wise feed-forward network (MLP) and the core ISTVTBlock that stacks temporal attention, spatial attention, and the MLP with residual connections.

A. Feed-Forward Network (MLP)

Each transformer block contains a two-layer position-wise MLP applied independently to each token. The hidden dimension is expanded by a factor of `mlp_ratio=4` relative to the embedding dimension, yielding an intermediate dimension of 2048 for our 512-dimensional embedding space. The expansion layer is followed by a GELU activation and dropout, before projecting back down to the original embedding dimension with a second dropout applied at the output. This design follows the standard transformer FFN formulation, with GELU preferred over ReLU for its smoother gradient properties in attention-based architectures.

B. ISTVT Transformer Block

The ISTVTBlock constitutes the fundamental building unit of the ISTVT encoder. Each block processes a spatiotemporal token sequence of shape $(B, T, N+1, D)$ through three sequential sub-layers, each preceded by a dedicated LayerNorm and followed by a residual connection — following the Pre-LN transformer formulation, which has been shown to improve training stability over the original Post-LN design. The three sub-layers are applied in the following order:

Temporal Attention with Residual: The input is first normalized via `norm1` and passed through the TemporalAttention module, which applies the self-subtract mechanism to Q and K projections before computing multi-head attention across the temporal dimension. The output is added back to the input via a residual connection, allowing the block to refine temporal inconsistency representations without disrupting the original token values.

Spatial Attention with Residual: The updated sequence is normalized via `norm2` and passed through the SpatialAttention module, which computes standard multi-head self-attention independently within each frame across the 65 spatial tokens. The residual connection again preserves the input signal, allowing spatial artifact features to be additively incorporated.

Feed-Forward Network with Residual: Finally, the sequence is normalized via `norm3` and passed through the MLP, which applies the same transformation independently to each token position. The residual connection completes the block, and the output maintains the original shape of $(B, 6, 65, 512)$, allowing identical blocks to be stacked in sequence.

C. Full Transformer Stack

Six ISTVTBlock modules are stacked sequentially in the full ISTVT model. Each block alternates between capturing temporal inconsistencies via the self-subtract-augmented temporal attention and spatial artifacts via frame-wise spatial attention, before refining token representations through the MLP. This interleaved design ensures that both modalities of deepfake evidence — unnatural temporal motion patterns and intra-frame visual artifacts — are jointly modeled across all layers of the transformer stack.

V. TRAINING PROTOCOL AND OPTIMIZATION

The training strategy was designed to bridge the theoretical framework of the original ISTVT paper with the practical hardware constraints of our implementation. Every hyperparameter and data-handling decision was made to ensure the model learned genuine spatiotemporal artifacts rather than memorizing dataset biases.

A. Objective Function

Since deepfake detection is a binary classification task, the network was optimized using `CrossEntropyLoss` with explicit class weights to handle dataset imbalance. The dataset contains 6000 fake clips and 4500 real clips, making the fake class the majority. To prevent the model from developing a bias toward predicting fake, the real class (class 0) was assigned a weight of $6000/4500 \approx 1.33$ while the fake class (class 1) retained a weight of 1.0. This weighted loss ensures that misclassifications of real videos are penalized proportionally more, encouraging balanced precision and recall across both classes.

The loss is formulated as:

$$\mathcal{L} = - \sum_{c \in \{0,1\}} w_c \cdot y_c \cdot \log(p_c)$$

where w_c is the class weight, y_c is the one-hot ground-truth label, and p_c is the predicted softmax probability for class c .

B. Optimization Strategy and Hyperparameters

Transformer architectures benefit from optimizers that handle sparse gradients and adapt learning rates per parameter. To address this, the network was optimized using AdamW with a learning rate of 1×10^{-4} and weight decay of 1×10^{-4} . AdamW decouples weight decay from the gradient update step, preventing the optimizer from inadvertently scaling the regularization term with the gradient magnitude, which is a known issue with standard Adam. No explicit warmup schedule was implemented; AdamW’s adaptive per-parameter learning rates provide implicit stability during the early stages of training. The model was trained for 30 epochs with a batch size of 8. A dropout rate of 0.1 was applied throughout the transformer blocks and classification head. The random seed was fixed to 42 across all components (Python, NumPy, PyTorch, CUDA) to ensure full reproducibility. All hyperparameters are consolidated in Table III.

TABLE III: Training Hyperparameters

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-4}
Batch size	8
Epochs	30
Dropout	0.1
MLP ratio	4 (hidden dim = 2048)
Embed dim	512
Attention heads	8
Transformer blocks	6
Train / Val / Test	70% / 15% / 15%
Random seed	42

C. Data Splitting and Leakage Prevention

A common pitfall in deepfake detection research is data leakage, where clips from the same source video appear in both the training and test sets, allowing the model to exploit identity-specific features such as background lighting or facial structure rather than learning genuine forgery artifacts. To prevent this, the dataset was partitioned using a strict video-level split: 70% training, 15% validation, and 15% test. All clips derived from the same source video were assigned exclusively to one partition, ensuring that the validation and test sets contain entirely unseen identities and environments. This provides a true measure of the model’s ability to generalize to novel deepfake instances.

D. Hardware-Aware Batching

Modifying the input resolution to 128×128 and the embedding dimension to 512 was a necessary compromise to fit the spatial-temporal computations within the available VRAM on RTX 2050 and

Apple M3 hardware. Additionally, freezing the Xception backbone parameters further reduced memory overhead during backpropagation, as gradients were not computed for the backbone parameters. This allowed a stable batch size of 8, which provides sufficient gradient diversity for the weighted `CrossEntropyLoss` to receive balanced real and fake samples in each backward pass.

VI. MODEL INTERPRETABILITY

A. Interpretability Framework

Although transformer-based models achieve strong performance in deepfake detection, understanding the reasoning behind their predictions is equally important. To provide qualitative insight into the learned representations, an interpretability module was implemented based on Gradient Attention Rollout. This method utilizes the trained ISTVT model to extract attention matrices and their corresponding gradients from every transformer block, identifying the spatial regions and temporal frames that contribute most significantly to the final prediction.

This differs from the original paper’s use of Layer-wise Relevance Propagation (LRP) / Deep Taylor Decomposition, which requires modifying the backward pass of every individual layer. Gradient Attention Rollout was chosen as a practical alternative that achieves similar localization quality without requiring layer-specific relevance rules, making it significantly easier to implement on top of a standard PyTorch transformer without modifying the core architecture.

B. Spatial Attention Visualization

During inference, a forward pass is performed on a test clip followed by backpropagation with respect to the predicted class score. The attention matrices from each spatial transformer block are combined with their gradients to obtain gradient-weighted attention maps. These maps are recursively aggregated across all transformer layers using the attention rollout procedure, producing a cumulative attention representation. The resulting spatial attention mask is resized to the original input resolution (128×128) and overlaid on each of the six facial frames using a color heatmap. Regions receiving higher attention indicate areas that have a greater influence on the model’s classification decision.

C. Temporal Attention Analysis

In addition to spatial localization, temporal attention analysis was performed to investigate the contribution of individual frames within a clip. Gradient-weighted temporal attention matrices were extracted from every temporal transformer block and aggregated across all layers. The resulting visualization illustrates the distribution of attention over the six-frame sequence, providing insight into how the model identifies temporal inconsistencies introduced by facial manipulations. Together, the spatial and temporal attention visualizations demonstrate that the proposed ISTVT implementation captures both localized forgery artifacts and inter-frame inconsistencies, thereby improving the interpretability of the model while validating its underlying temporal-to-spatial attention mechanism.

VII. RESULT

A. Quantitative Evaluation

The performance of the proposed ISTVT model was evaluated on the held-out test set using standard binary classification metrics, namely accuracy, precision, recall, and F1-score. The trained model achieved a test accuracy of 98.10%, demonstrating its effectiveness in distinguishing between authentic and manipulated facial videos. The high precision and recall values further indicate that the model produces very few false positives and false negatives, respectively. The quantitative evaluation results are summarized in Table IV.

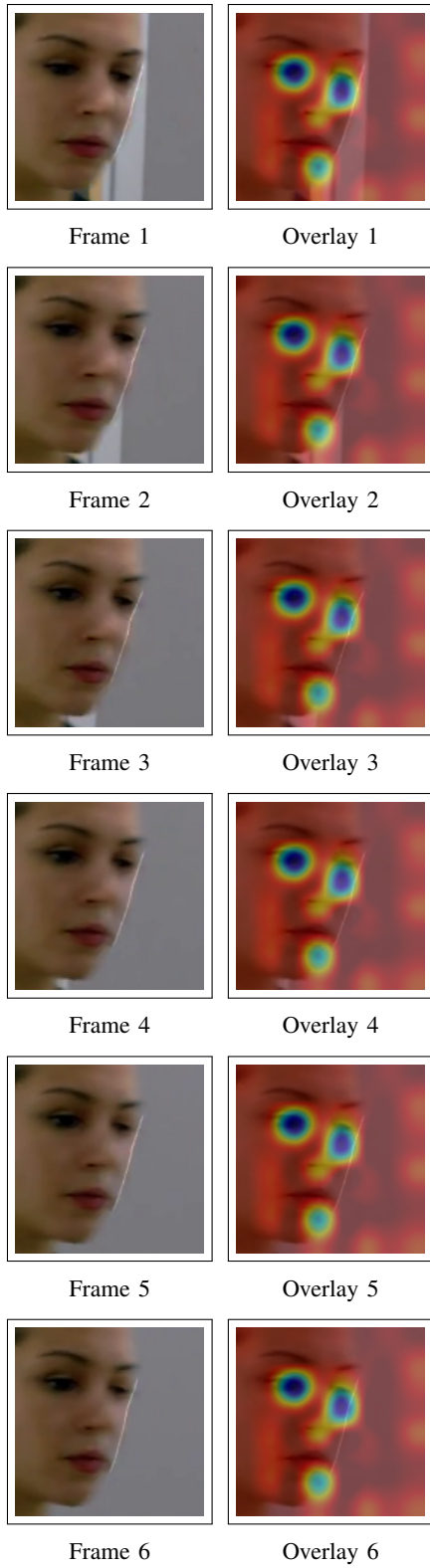


Fig. 5: Spatial attention visualization using Gradient Attention Rollout. The original frames and corresponding attention overlays are shown for a representative six-frame clip.

B. Confusion Matrix Analysis

Figure 7 illustrates the confusion matrix obtained on the test dataset. Out of **1575** test samples, the model correctly classified **652** real videos and **893** manipulated videos, while misclassifying only **23** real videos and **7** manipulated videos. The low number of classification errors demonstrates that the proposed ISTVT architecture effectively captures both spatial forgery artifacts and temporal inconsistencies, leading to robust performance across both classes.

C. Discussion

The experimental results demonstrate that the proposed implementation successfully reproduces the core ideas of the ISTVT architecture while maintaining high classification performance under

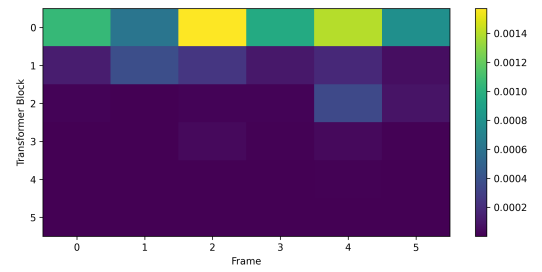


Fig. 6: Temporal attention visualization illustrating the attention assigned to each frame across successive transformer blocks

TABLE IV: Performance of the Proposed ISTVT Model on the Test Set

Metric	Value
Accuracy	98.10%
Precision	97.49%
Recall	99.22%
F1-Score	98.35%

the reduced computational setting adopted in this work. Despite employing 128×128 facial inputs, a 512-dimensional embedding space, 6 transformer blocks instead of 12, and training for only 30 epochs on RTX 2050/M3 hardware, the model achieved strong quantitative performance with 98.10% accuracy, 97.49% precision, 99.22% recall, and 98.35% F1-score on the FF++ C23 test set.

This indicates that the proposed modifications substantially reduce computational requirements without significantly compromising detection capability. The frozen Xception backbone preserved low-level texture features while keeping the trainable parameter count manageable. The weighted `CrossEntropyLoss` effectively handled the class imbalance between 4500 real and 6000 fake clips, as evidenced by the balanced precision and recall values. The interpretability analysis presented in Section VI provides qualitative evidence that the model focuses on meaningful facial regions and temporal inconsistencies when performing deepfake classification.

VIII. CONCLUSION

In this project, we implemented the Interpretable Spatial-Temporal Video Transformer (ISTVT) for deepfake detection using a subset of the FaceForensics++ C23 dataset. The implementation reproduces the two core contributions of the original paper: decomposed spatial-temporal self-attention with a self-subtract mechanism, and separate spatial and temporal interpretability via Gradient Attention Rollout.

Despite operating under significant computational constraints — reduced input resolution (128×128), smaller embedding dimension

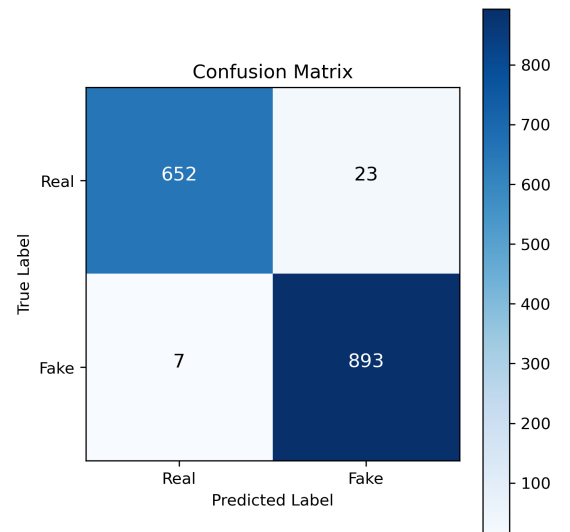


Fig. 7: Confusion Matrix

(512), fewer transformer blocks (6 vs. 12), and consumer-grade hardware — the implementation achieved 98.10% accuracy on the FF++ C23 test set, demonstrating that the core architectural ideas transfer effectively even in a resource-constrained setting.

The self-subtract mechanism proved critical for temporal reasoning by forcing attention to focus on inter-frame change rather than static content. The decomposed attention design reduced computational complexity from $\mathcal{O}(T^2 H^2 W^2)$ to $\mathcal{O}(T^2 + H^2 W^2)$, making video-level deepfake detection practical without specialized hardware.

Future work could explore testing on cross-dataset benchmarks such as Celeb-DF and DFDC, training with higher input resolution, unfreezing the backbone after initial convergence, and comparing against baseline models such as frame-only Xception to quantify the contribution of the temporal attention branch.

IX. LIMITATIONS

While the proposed implementation achieves strong performance on the FF++ C23 test set, several limitations should be noted.

First, the reduced input resolution of 128×128 may cause the model to miss fine-grained texture artifacts that are only visible at higher resolutions, particularly for subtle manipulation methods such as NeuralTextures. Second, the Xception backbone was fully frozen during training, which prevents the feature extractor from adapting to deepfake-specific low-level patterns beyond what was learned on ImageNet. Third, the implementation was evaluated exclusively on the FF++ C23 dataset, and no cross-dataset generalization experiments were conducted on Celeb-DF or DFDC due to computational constraints. Fourth, no comparison against baseline models such as frame-only Xception or C3D was performed, making it difficult to isolate the contribution of the temporal attention branch quantitatively. Finally, training for 30 epochs with a batch size of 8 may not allow full convergence compared to the original paper’s 100-epoch setting with 4 Tesla V100 GPUs.

X. REFERENCE

- 1) ISTVT: Interpretable Spatial-Temporal Video Transformer for Deepfake Detection

★ ★ ★